

```
import numpy as np
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge, LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import train_test_split
from scipy import stats
from sklearn.decomposition import PCA, FastICA
```

```
df = pd.read_csv("advanced.csv")
print(df.shape)
df.head()
```

```
(1599, 12)
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6

```
z_scores = np.abs(stats.zscore(df, nan_policy='omit'))
threshold = 3
outlier_mask = (z_scores > threshold)
rows_with_outliers = outlier_mask.any(axis=1)
num_outlier_rows = rows_with_outliers.sum()

print("Rows with outliers:", num_outlier_rows)
```

```
Rows with outliers: 148
```

```
outliers_per_column = outlier_mask.sum(axis=0)

print("Outliers per column:")
print(outliers_per_column)
```

```
Outliers per column:
fixed acidity      12
volatile acidity   10
citric acid         1
residual sugar     30
chlorides          31
free sulfur dioxide 22
total sulfur dioxide 15
density           18
pH                 8
sulphates         27
alcohol            8
quality           10
dtype: int64
```

```
percentage = (num_outlier_rows / len(df)) * 100
print(f"Outlier rows: {percentage:.2f}%")
```

```
Outlier rows: 9.26%
```

Now we are removing the outliers

```
df_clean = df[~rows_with_outliers]

Xr = df_clean.drop("quality", axis=1)
yr = df_clean["quality"]
```

```
X_train, X_test, y_train, y_test = train_test_split( Xr, yr, test_size=0.2, random_state=42)
```

k=100

```
scores_pca=[]
for i in range (1,k):
    Xr_pca_knn = Pipeline([
        ("scaler", StandardScaler()),
        ("pca", PCA(n_components=0.95)),
        ("knn", KNeighborsClassifier(n_neighbors=i))
    ])
    Xr_pca_knn_score = cross_val_score(Xr_pca_knn,Xr,yr,cv=5).mean()
    scores_pca.append(Xr_pca_knn_score)

print("the best output {result}, came with {k} neighbours".format(result=max(scores_pca), k=scores_pca.index(max(scores_pca))+1))
```

the best output 0.579630287948809, came with 65 neighbours

```
scores_ica=[]
for i in range (1,k):
    Xr_ica_knn = Pipeline([
        ("scaler", StandardScaler()),
        ("ica",
            FastICA(
                n_components=10,
                whiten='unit-variance',
                random_state=42,
            )
        ),
        ("knn", KNeighborsClassifier(n_neighbors=i))
    ])
    Xr_ica_knn_score = cross_val_score(Xr_ica_knn,Xr,yr,cv=5).mean()
    scores_ica.append(Xr_ica_knn_score)

print("the best output {result}, came with {k} neighbours".format(result=max(scores_ica), k=scores_ica.index(max(scores_ica))+1))
```

```
scores=[]
for i in range (1,k):
    Xr_knn = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=i))
    ])
    Xr_knn_score = cross_val_score(Xr_knn,Xr,yr,cv=5).mean()
    scores.append(Xr_knn_score)

print("the best output {result}, came with {k} neighbours".format(result=max(scores), k=scores.index(max(scores))+1))
```

the best output 0.5837492593909231, came with 84 neighbours

Now Capping the outlier

```
Xc = df.drop("quality", axis=1)
yc = df["quality"]

for col in Xc.columns:
    mean = Xc[col].mean()
    std = Xc[col].std()

    lower = mean - 3 * std
    upper = mean + 3 * std

    Xc[col] = Xc[col].clip(lower, upper)
```

```
X_train, X_test, y_train, y_test = train_test_split( Xc, yc, test_size=0.2, random_state=42)
```

k=100

```
scores_pca=[]
for i in range (1,k):
    Xr_pca_knn = Pipeline([
        ("scaler", StandardScaler()),
        ("pca", PCA(n_components=0.95)),
```

```

        ("knn", KNeighborsClassifier(n_neighbors=i))
    ])
    Xr_pca_knn_score = cross_val_score(Xr_pca_knn,Xr,yr,cv=5).mean()
    scores_pca.append(Xr_pca_knn_score)

print("the best output {result}, came with {k} neighbours".format(result=max(scores_pca), k=scores_pca.index(max(scores_pca))+1))

```

the best output 0.579630287948809, came with 65 neighbours

```

scores_ica=[]
for i in range (1,k):
    Xr_ica_knn = Pipeline([
        ("scaler", StandardScaler()),
        ("ica",
         FastICA(
             n_components=10,
             whiten='unit-variance',
             random_state=42,
         )
        ),
        ("knn", KNeighborsClassifier(n_neighbors=i))
    ])
    Xr_ica_knn_score = cross_val_score(Xr_ica_knn,Xr,yr,cv=5).mean()
    scores_ica.append(Xr_ica_knn_score)

print("the best output {result}, came with {k} neighbours".format(result=max(scores_ica), k=scores_ica.index(max(scores_ica))+1))

```

```

scores=[]
for i in range (1,k):
    Xr_knn = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=i))
    ])
    Xr_knn_score = cross_val_score(Xr_knn,Xr,yr,cv=5).mean()
    scores.append(Xr_knn_score)

print("the best output {result}, came with {k} neighbours".format(result=max(scores), k=scores.index(max(scores))+1))

```

the best output 0.5837492593909231, came with 84 neighbours

Start coding or [generate](#) with AI.